

BACKEND EXPLANATION DOCUMENT

Smart Gym Management System

Student Name: Sara Abid – bscs24112

Student Name: Ammara Khan – bscs24133

Group Number: 14

1.1 High-Level Architecture

The Smart Gym Management System follows a three-tier architecture.

Frontend Layer

Flutter Web Frontend (User Interface : Phase 3)

- Member Dashboard
- Admin Panel
- Trainer Portal

Communication Layer

HTTP / HTTPS

REST API Calls

JSON Data Format

Backend Layer

Flask Backend API (Business Logic)

Components inside backend:

- JWT Authentication Middleware
- Role-Based Access Control (RBAC)
- API Endpoints (30+ routes)

Main API categories:

- Authentication
- Members
- Trainers
- Staff
- Admin
- General (Public)

Other backend systems:

- Transaction Management (BEGIN / COMMIT / ROLLBACK)

Database Communication:

- psycopg2 PostgreSQL driver
- Connection Pool (1–20 connections)

- Parameterized SQL queries

Database Layer

PostgreSQL Database (Data Layer)

Database contains:

- 15 normalized tables (3NF)
- 3 strategic indexes
- 3 business rule triggers
- 3 analytical views

1.2 Technology Stack Justification

Component	Technology	Reason
Backend Framework	Flask (Python)	Lightweight and easy for understanding as we had little knowledge of it.
Database	PostgreSQL	Strong ACID compliance and trigger support
Authentication	JWT (JSON Web Tokens)	Stateless and scalable
Password Hashing	bcrypt	Secure salted hashing
Database Driver	psycopg2	Native PostgreSQL adapter
API Style	REST	Standard and frontend-friendly

2. REQUEST LIFECYCLE

2.1 Typical API Request Flow

Example: A member books a class.

Step 1: Client makes Request

POST /api/v1/members/book-class

Headers:

Authorization: Bearer TOKEN

Content-Type: application/json

Body:

schedule_id = 7

Step 2 : Flask Receives Request

Flask routes the request to the members blueprint.

The endpoint executed is the book_class() function.

Step 3: Middleware Layer

The token_required decorator runs.

Actions performed:

- Extract JWT token from the Authorization header
- Verify token signature using SECRET_KEY
- Decode user information
- Check token expiration

If it's valid then the request continues otherwise, 401 Unauthorized.

Step 4: RBAC Layer

The role_required decorator runs.

It checks:

current_user role == member

If true then the request proceeds

If false then error 403 Forbidden is generated.

Step 5: Business Logic

The book_class function executes.

Operations:

- Get database connection
- Validate schedule ID
- Check duplicate booking
- Insert new booking into class_bookings

A trigger runs automatically:
prevent_class_overbooking()

Trigger tasks:

First check class capacity and then confirm booking if space exists. Otherwise, just add to the waitlist and update capacity if confirmed.

Transaction is committed.

Step 6: Database Execution

PostgreSQL executes:

- INSERT statement
- Trigger logic
- Returns booking details

Step 7: Response Formatting

Flask returns JSON:

success = true

message = Class booked successfully

Booking object:

- booking_id
- status
- waitlist position

Step 8 : Client Receives Response

HTTP Status: 201 Created

Frontend receives JSON response.

3. AUTHENTICATION AND AUTHORIZATION

3.1 JWT Token Generation Flow

When a user logs in:

1. User submits email and password.
2. Backend queries the users table
3. Password is verified using bcrypt
4. JWT token is generated
5. Token is returned to client

JWT payload contains:

- user_id
- email
- role
- expiration time
- issued time

Token is valid till **24 hours**.

JWT structure is divided into three parts: header, payload, and signature.

3.2 Token Verification Middleware

The token required decorator performs:

1. Extract token from the Authorization header
2. Decode token using SECRET_KEY
3. Validate expiration
4. Extract user details

Errors handled:

ExpiredSignatureError refers that token expired

InvalidTokenError refers Invalid token

If valid, the request continues.

3.3 Role Based Access Control

The role required by the decorator ensures only authorized roles can access endpoints.

Example access rules:

Member-only endpoint allows that only members can access the dashboard.

Admin-only endpoint allows that only admins can view all users.

Mixed access endpoint allows only trainers and admins allowed.

3.4 Security Features Implemented

Security Feature	Implementation	Purpose
Password Hashing	bcrypt	Protect user passwords

Token Expiry	24 hours	Reduce stolen token risk
Stateless Auth	JWT	Scalable authentication
Role Enforcement	Middleware checks	Prevent privilege abuse
SQL Injection Prevention	Parameterized queries	Protect database
CORS Control	Flask-CORS	Restrict external access

4. TRANSACTION MANAGEMENT

4.1 Transaction Scenario 1 (Membership Purchase)

Business requirement:

Multiple database operations must succeed together:

1. Create payment record
2. Create membership record
3. Assign locker

If any step fails, **all operations must rollback.**

Transaction steps:

BEGIN TRANSACTION

1. Retrieve membership tier details
2. Insert payment record
3. Insert membership record
4. Assign locker if tier is Premium or VIP

COMMIT if successful.

If any error occurs:

ROLLBACK is executed.

ACID Properties satisfied

Atomicity: All operations succeed together.

Consistency: Database constraints enforce valid data.

Isolation: SELECT FOR UPDATE prevents race conditions.

Durability: Committed data persists even after crashes.

4.2 Transaction Scenario 2 (Class Booking with Trigger)

Process:

1. Member requests class booking
2. Backend checks duplicate bookings
3. Insert booking record
4. Trigger handles capacity management

Trigger actions:

- If class is full then add to waitlist
- Otherwise confirm booking
- Update class capacity

Transaction is committed after insertion.

ACID Properties satisfied

Atomicity: Insert and trigger execute together.

Consistency: Capacity never exceeds maximum.

Isolation: PostgreSQL row locks prevent conflicts.

Durability: Committed booking persists.

4.3 Concurrency Handling

Problem:

Two users are booking the last available seat.

Solution:

Use **SELECT FOR UPDATE** row locking.

Result:

- First transaction locks the row
- Second transaction waits
- Prevents overbooking

5. DATABASE INTERACTION STRATEGY

5.1 Raw SQL

Use Raw SQL with `psycopg2`

Reasons:

Factor	Raw SQL
Transaction Control	Full control
Performance	Faster
Trigger Integration	Direct
SQL Learning	Stronger
Complex Queries	Fully supported

5.2 Connection Pooling

Database connections are expensive to create.

Solution:

Use **connection pooling**.

Configuration:

Minimum connections: 1

Maximum connections: 20

Process:

1. Request gets connection from pool
2. Query executes
3. Connection returned to pool

Benefits:

- Faster performance
- Controlled database load
- Efficient resource usage

5.3 SQL Injection Prevention

Unsafe approach:

Building queries using string concatenation.

Safe approach:

Use **parameterized queries** with placeholders.

Example concept:

```
SELECT * FROM users WHERE email = parameter
```

The database safely escapes user input.

This prevents malicious SQL execution.

6. KEY DESIGN DECISIONS AND TRADEOFFS

6.1 Flask

Chosen: Flask

Reasons:

- Lightweight framework
- Flexible structure
- Easier for us as we had strong Python familiarity.

6.2 JWT vs Session Authentication

Chosen: JWT

Advantages:

- Stateless authentication
- Horizontal scalability
- Frontend friendly
- Industry standard

Tradeoff:

Tokens cannot easily be revoked before expiry.

6.3 API Versioning

Approach used:

URL versioning

Example:

/api/v1/members/dashboard

Benefits:

- Clear version separation
- Easy upgrades
- Backward compatibility

6.4 Error Handling Philosophy

Development environment:

Return detailed errors for debugging.

Production environment:

Return generic error messages.

This protects internal system information.

7. CONCLUSION

This backend API demonstrates strong backend development concepts through a RESTful design with 30+ endpoints. It implements a secure authentication system using JWT for token-based authentication, bcrypt for password hashing, and role-based access control. The system ensures reliable database operations using transactions (BEGIN, COMMIT, ROLLBACK) to maintain data integrity. It also integrates the database efficiently using raw SQL, triggers, views, and indexes, along with strong error handling and rollback protection.